WEB TASARIMI VE PROGRAMLAMA

LAB-3

Modern CSS · Responsive Layout (Flex/Grid) · Tasarım Sistemi

İçindekiler

1 LAB-3 Öğrenme Hedefleri **3**

2 Modern CSS ve Responsive Tasarıma Giriş **3**

3 Mobile-First Yaklaşımı ve Breakpoints **4**
3.1 Mobile-First Nedir? 4
3.2 Breakpoint'ler 6

4 CSS Değişkenleri ve Design Tokens **6**
4.1 CSS Custom Properties Nedir? 6
4.2 Design Tokens Kavramı 8

5 Fluid Typography **9**

6 Flexbox Temelleri **10**

7 CSS Grid Temelleri **13**

8 Uygulama: Flex ile Nav ve Toolbar **14**

9 Uygulama: Grid ile Kart Düzeni **17**

10 Uygulama: Portföy Sayfasını 3 Breakpoint'e Göre Responsive Yap **19**

11 CSS Kararları Notu **23**

12 Git İş Akışı ve Teslim **24**

13 Teslim Kontrol Listesi ve Değerlendirme Rubriği **25**
13.1 Değerlendirme Rubriği 25

14 Kendini Test Et **26**

15 Ek Kaynaklar ve İleri Okuma **26**

1 LAB-3 Öğrenme Hedefleri

Önceki laboratuvarlarda geliştirme ortamını kurmuş (LAB-1) ve semantik, erişilebilir bir HTML iskeleti oluşturmuştun (LAB-2). Artık yapı sağlam; şimdi sıra bu yapıyı **her ekran boyutunda güzel ve kullanılabilir** kılmaya geldi.

Bu laboratuvarda CSS'in modern layout araçlarını (Flexbox ve Grid), responsive tasarım ilkelerini (mobile-first, breakpoints, fluid typography) ve tasarım sisteminin temellerini (CSS değişkenleri / design tokens) öğreneceksin. Ayrıca LAB-2'de oluşturduğun portföy sayfasını **3 farklı ekran boyutuna** (mobil, tablet, masaüstü) uyumlu hale getireceksin.

Bu laboratuvarın sonunda aşağıdaki becerilere sahip olacaksın:

1. **Mobile-first yaklaşımla CSS yazmak:** Önce en küçük ekran için tasarlayıp, büyük ekranlara doğru `min-width` media query'leri ile genişletmek.
2. **Breakpoint'leri doğru belirlemek:** Mobil (< 640px), tablet (640–1024px) ve masaüstü (> 1024px) kırılım noktalarını anlamak ve uygulamak.
3. **Fluid typography kullanmak:** `clamp()` fonksiyonu ile yazı boyutlarının ekran genişliğine göre akıcı şekilde ölçeklenmesini sağlamak.
4. **CSS değişkenleri (custom properties) tanımlamak:** Renk, spacing, border-radius gibi tasarım kararlarını merkezi bir yerde (design tokens) toplamak.
5. **Flexbox ile tek boyutlu düzen kurmak:** Navigasyon çubuğu, toolbar ve hizalama işlemlerini Flex ile çözmek.
6. **CSS Grid ile iki boyutlu düzen kurmak:** Kart düzeni (card layout), galeri ve sayfa genel layout'unu Grid ile oluşturmak.
7. **Portföy sayfasını responsive yapmak:** LAB-2'deki sayfayı 3 breakpoint'e uyumlu hale getirmek ve her biri için ekran görüntüsü almak.

Benzetme ile Anlama

LAB-1'de inşaat aletlerini kurmuş, LAB-2'de binanın betonarme iskeletini dökmüştük. Şimdi **iç mimariyi** yapıyoruz: Odaların düzeni, mobilyaların yerleşimi, kapı ve pencerelerin boyutu… Üstelik bu bina "akıllı" olacak — oda sayısı ve genişliği, içindeki kişi sayısına göre otomatik ayarlanacak. Responsive tasarım tam olarak budur: İçerik, bulunduğu kaba göre kendini uyarlar.

Sırada Ne Var?

Hedeflerimiz belirlendi. İlk adım olarak "neden responsive tasarım?" sorusunu cevaplayalım ve mobile-first felsefesini anlayalım.

2 Modern CSS ve Responsive Tasarıma Giriş

2024 verilerine göre web trafiğinin **%60'ından fazlası** mobil cihazlardan geliyor. Kullanıcılar aynı siteye 4 inçlik telefondan, 10 inçlik tabletten ve 27 inçlik monitörden erişiyor. Tek bir sabit genişlikte tasarım yapmak artık kabul edilemez.

Teorik Arka Plan

Responsive Web Tasarım (RWD) kavramı, web sayfasının **her ekran boyutunda** okunabilir, kullanılabilir ve estetik olmasını hedefler. Üç temel bileşeni vardır:

1. **Esnek ızgaralar (Fluid grids):** Piksel yerine yüzde (%), `fr`, `vw` gibi göreceli birimler kullanmak.
2. **Esnek medya:** Görsellerin `max-width: 100%` ile kapsayıcısına sığması.
3. **Media query'ler:** Ekran genişliğine göre CSS kurallarını değiştirmek.

Viewport meta etiketi responsive tasarımın ön koşuludur. HTML `<head>` bölümünde mutlaka bulunmalıdır:

```
<meta name="viewport"
      content="width=device-width, initial-scale=1.0" />
```

Bu etiket olmadan mobil tarayıcılar sayfayı masaüstü genişliğinde render edip küçültür — kullanıcı parmak ucuyla yakınlaştırmak zorunda kalır.

Gerçek Hayatta Karşılığı

Bir gazete düşün. Aynı gazete A3 kağıda 6 sütunla basılırken, cep boyutunda 1 sütunla basılır. İçerik aynıdır ama **düzen** (layout) farklıdır. Responsive tasarım web için tam olarak budur.

Sırada Ne Var?

Responsive tasarımın "ne" ve "neden"ini anladık. Şimdi en temel stratejiyi öğrenelim: **mobile-first** yaklaşımı.

3 Mobile-First Yaklaşımı ve Breakpoints

CSS yazarken iki farklı strateji vardır: **Desktop-first** (büyükten küçüğe) veya **Mobile-first** (küçükten büyüğe). Modern web geliştirmede **mobile-first** standart kabul edilir.

3.1 Mobile-First Nedir?

Teorik Arka Plan

Mobile-first yaklaşımda CSS'i önce **en küçük ekran** (mobil telefon) için yazarsın. Ardından `min-width` media query'leri ile daha büyük ekranlar için kurallar eklersin.

Mobile-first (doğru):

```
/* Varsayilan: Mobil (dar ekran) */
.container {
  display: flex;
  flex-direction: column;
  padding: 1rem;
}

/* Tablet ve uzeri (640px+) */
@media (min-width: 640px) {
```

```
  .container {
    flex-direction: row;
    padding: 2rem;
  }
}

/* Masaustu (1024px+) */
@media (min-width: 1024px) {
  .container {
    max-width: 1200px;
    margin: 0 auto;
  }
}
```

Desktop-first (eski yöntem):

```
/* Varsayilan: Masaustu */
.container {
  display: flex;
  flex-direction: row;
  max-width: 1200px;
  margin: 0 auto;
}

/* Tablet ve alti (1024px-) */
@media (max-width: 1024px) {
  .container { max-width: 100%; }
}

/* Mobil (640px-) */
@media (max-width: 640px) {
  .container { flex-direction: column; }
}
```

Derinlemesine Anlama

Mobile-first neden daha iyidir?

- **Performans:** Mobil cihazlar yavaş ağ bağlantısına sahip olabilir. Mobile-first'te mobil kullanıcılar yalnızca temel CSS'i indirir; fazla media query kuralları yüklenmez. Desktop-first'te ise mobil cihaz tüm masaüstü kurallarını indirip ardından "üzerine yazılan" kuralları da indirir.
- **İçerik önceliği:** Dar ekranda en önemli içeriğe odaklanırsın. Büyük ekrana geçtikçe **ekleme** yaparsın (column → row, daha geniş padding, sidebar ekleme). "Ekleme" stratejisi, "çıkarma" stratejisinden daha kolay yönetilir.
- **Progresif geliştirme:** Temel deneyim herkese sunulur; daha yetenekli cihazlar zenginleştirilmiş deneyim alır.

3.2 Breakpoint'ler

Teorik Arka Plan

Breakpoint (kırılım noktası), layout'un değiştiği ekran genişliğidir. Sabit bir standart yoktur ama yaygın olarak kabul görmüş değerler şunlardır:

İsim	Genişlik	Hedef Cihaz
sm (small)	640px	Büyük telefonlar
md (medium)	768px	Tabletler (dikey)
lg (large)	1024px	Tabletler (yatay) / Küçük dizüstü
xl (extra large)	1280px	Masaüstü monitörler
2xl	1536px	Geniş monitörler

Bu LAB'da 3 ana breakpoint kullanacağız:

- **Mobil:** 0–639px (varsayılan, media query yok)
- **Tablet:** 640px ve üzeri (`@media (min-width: 640px)`)
- **Masaüstü:** 1024px ve üzeri (`@media (min-width: 1024px)`)

Dikkat – Sık Yapılan Hatalar

- Breakpoint'leri **cihaz modeline** göre değil, **içeriğe** göre belirle. "iPhone 15 genişliği 393px" diye sabit değer koymak yerine, içeriğin bozulduğu noktayı bul ve orada breakpoint koy.
- `max-width` ve `min-width` query'lerini karıştırma. Mobile-first'te tutarlı ol: her zaman `min-width` kullan.
- Çok fazla breakpoint eklemek kodu karmaşıklaştırır. Genelde 2–4 breakpoint yeterlidir.

Sırada Ne Var?

Mobile-first stratejiyi ve breakpoint mantığını kavradık. Şimdi tasarım kararlarını merkezi bir yerde toplamamızı sağlayan **CSS değişkenlerine (design tokens)** geçelim.

4 CSS Değişkenleri ve Design Tokens

Bir projede aynı mavi rengi 47 farklı yerde kullandığını düşün. Müşteri "mavi yerine lacivert olsun" dediğinde 47 satırı tek tek bulup değiştirmen gerekir. CSS değişkenleri bu sorunu köklü şekilde çözer.

4.1 CSS Custom Properties Nedir?

Teorik Arka Plan

CSS custom properties (özel özellikler), `--` ön ekiyle tanımlanan ve `var()` fonksiyonu ile kullanılan değişkenlerdir. Genelde `:root` seçicisinde (yani `<html>` elementinde) tanımlanır, böylece tüm sayfada kullanılabilir olur.

```
:root {
```

```
  /* --- Renk Paleti --- */
  --color-primary: #1E3A8A;
  --color-secondary: #2563EB;
  --color-accent: #7C3AED;
  --color-bg: #FFFFFF;
  --color-text: #1F2937;
  --color-muted: #6B7280;
  --color-surface: #F9FAFB;
  --color-border: #E5E7EB;
  --color-error: #DC2626;
  --color-success: #16A34A;

  /* --- Spacing (Bosluk) Skalasi --- */
  --space-xs: 0.25rem;   /*  4px */
  --space-sm: 0.5rem;    /*  8px */
  --space-md: 1rem;      /* 16px */
  --space-lg: 1.5rem;    /* 24px */
  --space-xl: 2rem;      /* 32px */
  --space-2xl: 3rem;     /* 48px */
  --space-3xl: 4rem;     /* 64px */

  /* --- Border Radius --- */
  --radius-sm: 4px;
  --radius-md: 8px;
  --radius-lg: 16px;
  --radius-full: 9999px;

  /* --- Font Aileleri --- */
  --font-sans: system-ui, -apple-system, sans-serif;
  --font-mono: 'Fira Code', 'Consolas', monospace;

  /* --- Font Boyutlari (Fluid) --- */
  --text-sm: clamp(0.8rem, 0.75rem + 0.25vw, 0.9rem);
  --text-base: clamp(1rem, 0.9rem + 0.5vw, 1.125rem);
  --text-lg: clamp(1.125rem, 1rem + 0.6vw, 1.35rem);
  --text-xl: clamp(1.25rem, 1rem + 1.2vw, 1.75rem);
  --text-2xl: clamp(1.5rem, 1rem + 2vw, 2.5rem);
  --text-3xl: clamp(2rem, 1.2rem + 3vw, 3.5rem);

  /* --- Golge --- */
  --shadow-sm: 0 1px 2px rgba(0,0,0,0.05);
  --shadow-md: 0 4px 6px rgba(0,0,0,0.07);
  --shadow-lg: 0 10px 15px rgba(0,0,0,0.1);

  /* --- Gecis --- */
  --transition-fast: 150ms ease;
  --transition-normal: 300ms ease;
}
```

Kullanım:

```
body {
  font-family: var(--font-sans);
  font-size: var(--text-base);
  color: var(--color-text);
  background-color: var(--color-bg);
}

.card {
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  padding: var(--space-lg);
  box-shadow: var(--shadow-sm);
  transition: box-shadow var(--transition-normal);
}

.card:hover {
  box-shadow: var(--shadow-md);
}
```

4.2 Design Tokens Kavramı

Derinlemesine Anlama

Design tokens, tasarım kararlarının (renk, tipografi, boşluk, gölge vb.) **tek bir kaynakta** tanımlanıp tüm projede tutarlı kullanılmasını sağlayan bir mimari yaklaşımdır.
CSS değişkenleri, design token'ların CSS'teki uygulamasıdır. Büyük projelerde bu token'lar JSON formatında tanımlanıp otomatik araçlarla CSS, iOS (Swift), Android (Kotlin) gibi platformlara dönüştürülür. Bu LAB'da CSS düzeyinde kalacağız.
Token'ların faydaları:

- **Tutarlılık:** Tüm bileşenler aynı renk/spacing setini kullanır. "Bu buton niye farklı mavi?" sorusu ortadan kalkar.
- **Bakım kolaylığı:** Renk değiştirmek tek satırlık bir iştir; 47 yeri aramak gerekmez.
- **Tema desteği:** Dark mode eklemek, token değerlerini değiştirmek kadar kolaydır.
- **Ekip iletişimi:** Tasarımcı ve geliştirici aynı isimleri kullanır (`--color-primary`).

Uygulama Adımı

Uygulama-1: Design Token Dosyası Oluştur

1. Proje klasöründe `src/styles/` dizini oluştur (yoksa).
2. `src/styles/tokens.css` adında bir dosya oluştur.
3. Yukarıdaki `:root` bloğunu bu dosyaya yapıştır ve kendi renk tercihlerine göre düzenle.

4. Ana CSS dosyandan (`index.css` veya `App.css`) bu dosyayı import et:

```
@import './styles/tokens.css';
```

5. Mevcut CSS'inde sabit renk ve boyut değerleri varsa bunları `var(--...)` ile değiştirmeye başla.

Dikkat – Sık Yapılan Hatalar

- CSS değişkeni tanımlarken `--` ön eki zorunludur. `color-primary` geçersizdir; `--color-primary` doğrudur.
- `var()` fonksiyonuna **fallback** (yedek) değer verebilirsin: `var(--color-primary, #1E3A8A)`. Değişken tanımlı değilse yedek değer kullanılır.
- CSS değişkenleri `calc()` içinde de kullanılabilir: `calc(var(--space-md) * 2)`.
- `:root`'ta tanımlanan değişkenler tüm elementlerde kullanılabilir. Ama bir değişkeni belirli bir elemana özel tanımlamak da mümkündür (kapsam daraltma).

Sırada Ne Var?

Design token'larımızı tanımladık. Şimdi bu token'lardaki font boyutlarında gördüğümüz `clamp()` fonksiyonunu ve **fluid typography** kavramını detaylıca anlayalım.

5 Fluid Typography

Geleneksel yaklaşımda yazı boyutlarını media query'ler içinde breakpoint'lere göre değiştirirsin: Mobilde 14px, tablette 16px, masaüstünde 18px. Bu "adımlı" geçişler keskin ve ani olur. **Fluid typography** ise yazı boyutunun ekran genişliğine göre **akıcı ve kademesiz** şekilde ölçeklenmesini sağlar.

Teorik Arka Plan

`clamp()` fonksiyonu:

```
font-size: clamp(minimum, tercih-edilen, maksimum);
```

- **minimum:** Font boyutunun düşebileceği en küçük değer.
- **tercih edilen (preferred):** Genellikle `vw` (viewport width) birimi içerir ve ekran genişliğiyle orantılı şekilde değişir.
- **maksimum:** Font boyutunun çıkabileceği en büyük değer.

Örnek:

```
h1 {
  /* En az 2rem, en fazla 3.5rem,
     arada ekran genisligine gore akici */
  font-size: clamp(2rem, 1.2rem + 3vw, 3.5rem);
}

p {
  font-size: clamp(1rem, 0.9rem + 0.5vw, 1.125rem);
}
```

Nasıl çalışır?
Diyelim ki `clamp(1rem, 0.9rem + 0.5vw, 1.125rem)` kullanıyorsun:

- 320px genişlikte: `0.9rem + 0.5 × 3.2px = 0.9rem + 1.6px ≈ 1rem` → minimum sınırda.
- 768px genişlikte: `0.9rem + 0.5 × 7.68px = 0.9rem + 3.84px ≈ 1.14rem` → ama maksimum 1.125rem olduğu için 1.125rem'de durur.
- Aradaki tüm değerlerde akıcı geçiş olur.

Uygulama Adımı

Uygulama-2: Fluid Typography Uygula

1. `tokens.css` dosyandaki `--text-*` değişkenlerini kontrol et.
2. CSS'inde sabit `font-size` değerlerini `var(--text-*)` ile değiştir:

```
body { font-size: var(--text-base); }
h1   { font-size: var(--text-3xl); }
h2   { font-size: var(--text-2xl); }
h3   { font-size: var(--text-xl); }
```

3. Tarayıcı penceresini yavaşça genişletip daralt. Yazı boyutunun akıcı şekilde değiştiğini gözlemle. Media query yazmadan font boyutu uyum sağlıyor!

Dikkat – Sık Yapılan Hatalar

- `clamp()` içinde yalnızca `vw` kullanma (`font-size: 3vw`). Kullanıcı tarayıcıda zoom yaptığında `vw` değişmez ve yazı büyümez — erişilebilirlik ihlalidir. Her zaman `rem + vw` karışımı kullan.
- Minimum değeri `1rem`'in altına düşürme. 16px altındaki yazı boyutu çoğu kullanıcı için okunaksızdır.
- `line-height` için de göreceli birim kullan: `line-height: 1.6` (birimsiz) en güvenli seçenektir.

Sırada Ne Var?

Fluid typography ile metin boyutlarını akıcı hale getirdik. Şimdi layout'un temel taşına geçiyoruz: **Flexbox**. Tek boyutlu düzenlerin en güçlü aracı.

6 Flexbox Temelleri

Flexbox (Flexible Box Layout), elemanları **tek bir eksen** (yatay veya dikey) boyunca hizalamak ve dağıtmak için tasarlanmış CSS layout modülüdür. Navigasyon çubukları, toolbar'lar, form satırları ve basit hizalama işlemleri için idealdir.

Teorik Arka Plan

Flexbox'ta iki temel kavram vardır:

- **Flex container:** `display: flex` verilen eleman. Çocuklarını (flex item) kontrol eder.

- **Flex item:** Container'ın doğrudan çocuk elemanları. Container'ın kurallarına göre yerleşir.

Container özellikleri:

```
.flex-container {
  display: flex;

  /* Ana eksen yonu */
  flex-direction: row;       /* varsayilan: yatay */
  /* diger: column, row-reverse, column-reverse */

  /* Satirlama (tasmada ne olacak?) */
  flex-wrap: wrap;            /* tasma olursa alta sar */
  /* diger: nowrap (varsayilan), wrap-reverse */

  /* Ana eksen hizalama */
  justify-content: center;
  /* diger: flex-start, flex-end, space-between,
     space-around, space-evenly */

  /* Capraz eksen hizalama */
  align-items: center;
  /* diger: flex-start, flex-end, stretch,
     baseline */

  /* Elemanlar arasi bosluk */
  gap: 1rem;
}
```

Item özellikleri:

```
.flex-item {
  /* Buyume orani (fazla alan varsa ne kadar buyusun) */
  flex-grow: 1;

  /* Kucalme orani (alan yetersizse ne kadar kuculsun) */
  flex-shrink: 0;

  /* Baslangic boyutu */
  flex-basis: 200px;

  /* Kisayol: flex-grow flex-shrink flex-basis */
  flex: 1 0 200px;

  /* Tek bir item'in capraz eksen hizalamasi */
  align-self: flex-end;

  /* Siralama (DOM sirasi degismeden gorsel sira) */
  order: 2;
}
```

Benzetme ile Anlama

Flexbox'ı bir **elbise askısı** gibi düşün. Askı çubuğu (ana eksen) yatay uzanır; üzerindeki elbiseler (flex item'lar) bu çubuk boyunca sıralanır. `justify-content`, elbiseleri çubuğun soluna mı, ortasına mı yoksa eşit aralıklı mı dizileceğini belirler. `align-items` ise elbiselerin üst kenardan mı, ortadan mı yoksa alt kenardan mı asılacağını kontrol eder.

Uygulama Adımı

Uygulama-3: Flex ile Navigasyon Çubuğu
Portföy sayfandaki `<nav>` elemanını Flexbox ile yatay bir navigasyon çubuğuna dönüştür:

```
/* Navigasyon cubugu */
nav ul {
  display: flex;
  list-style: none;
  gap: var(--space-md);
  padding: 0;
  margin: 0;
}

/* Mobilde dikey yigin */
@media (max-width: 639px) {
  nav ul {
    flex-direction: column;
    align-items: center;
  }
}

/* Header icindeki hizalama: logo sol, nav sag */
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: var(--space-md) var(--space-xl);
}
```

Derinlemesine Anlama

`gap` vs `margin`: Hangisini Kullanmalı?
Elemanlar arası boşluk için eskiden her flex item'a `margin-right` verilirdi; son elemana `margin-right: 0` koymak gerekiyordu. `gap` özelliği bu sorunu kökten çözer: Yalnızca elemanlar **arasına** boşluk koyar, başa ve sona koymaz. Modern tarayıcıların tamamı Flexbox'ta `gap`'i destekler. Artık margin ile "son eleman" istisnası yazmaya gerek yoktur.

Sırada Ne Var?

Flexbox ile tek boyutlu düzenleri çözmeyi öğrendik. Peki iki boyutlu düzenler (satır **ve** sütun aynı anda) gerektiğinde ne yaparız? Cevap: **CSS Grid**.

7 CSS Grid Temelleri

CSS Grid, elemanları **hem satır hem sütun** boyunca aynı anda kontrol etmeyi sağlayan iki boyutlu layout sistemidir. Sayfa genel yapısı, kart düzenleri, galeri ızgaraları ve karmaşık layout'lar için idealdir.

Teorik Arka Plan

Grid'de üç temel kavram vardır:

- **Grid container:** `display: grid` verilen eleman.
- **Grid item:** Container'ın doğrudan çocuk elemanları.
- **Grid track:** Bir satır veya sütun. `grid-template-columns` ve `grid-template-rows` ile tanımlanır.

Temel Grid özellikleri:

```
.grid-container {
  display: grid;

  /* 3 esit sutun */
  grid-template-columns: 1fr 1fr 1fr;
  /* veya kisaca: */
  grid-template-columns: repeat(3, 1fr);

  /* Satir yukseklikleri */
  grid-template-rows: auto;

  /* Elemanlar arasi bosluk */
  gap: var(--space-lg);

  /* Hizalama (Flexbox'takine benzer) */
  justify-items: center;  /* yatay hizalama */
  align-items: center;    /* dikey hizalama */
}
```

`fr` birimi:
`fr` (fraction), kalan boş alanın **oransal** payıdır. `1fr 2fr 1fr` demek: 1. sütun 1 pay, 2. sütun 2 pay, 3. sütun 1 pay alır (toplam 4 pay).

`repeat()` ve `minmax()` fonksiyonları:

```
/* 3 esit sutun, her biri en az 200px */
grid-template-columns: repeat(3, minmax(200px, 1fr));

/* Otomatik sutun sayisi (responsive grid!) */
grid-template-columns:
  repeat(auto-fit, minmax(280px, 1fr));
```

`auto-fit` ile `minmax()`: Tarayıcı, kapsayıcı genişliğine göre kaç sütun sığacağını **otomatik** hesaplar. Ekran daraldığında sütun sayısı azalır, genişledikçe artar — **media query bile yazmadan responsive grid** elde edersin!

Benzetme ile Anlama

CSS Grid'i bir **Excel tablosu** gibi düşün. Satır ve sütunlardan oluşan bir ızgara (grid) tanımlarsın; elemanları bu ızgaranın hücrelerine yerleştirirsin. Bir eleman birden fazla hücreyi kaplayabilir (birleştirilmiş hücre gibi). Flexbox ise tek satırlık bir liste — yalnızca bir yönde düzen kurar.

Derinlemesine Anlama

Flexbox mı Grid mi? Hangisini Ne Zaman Kullanmalı?

- **Flexbox:** Tek boyutlu düzenler için. Navigasyon çubuğu, buton grubu, form satırı, dikey ortalama gibi **bir eksen** boyunca hizalama.
- **Grid:** İki boyutlu düzenler için. Sayfa layout'u, kart ızgarası, galeri, dashboard gibi **satır ve sütun** birlikte gerektiğinde.
- **Birlikte kullanım:** Grid ile genel sayfa layout'unu kurar, Grid hücreleri içinde Flexbox ile detay hizalama yaparsın. İkisi rakip değil, **tamamlayıcıdır**.

Pratik kural: "İçerik dışarı doğru büyüyor mu?" (Kaç eleman olacağı belirsiz, sarmalama gerekebilir) → **Flexbox**. "Dışarıdan içeriye kontrol ediyorum" (ızgara yapısı belirli) → **Grid**.

Sırada Ne Var?

Grid'in temellerini öğrendik. Şimdi Flexbox ve Grid'i uygulamalı olarak portföy sayfamıza entegre edeceğiz: Flex ile nav/toolbar, Grid ile kart düzeni.

8 Uygulama: Flex ile Nav ve Toolbar

Bu bölümde Flexbox'ı kullanarak portföy sayfanın header bölümünü responsive bir navigasyon çubuğuna dönüştüreceğiz.

Uygulama Adımı

Uygulama-4: Responsive Header ve Navigasyon

```
/* ===== HEADER ===== */
header {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  padding: var(--space-md) var(--space-lg);
  background-color: var(--color-surface);
  border-bottom: 1px solid var(--color-border);
  position: sticky;
  top: 0;
```

```
  z-index: 10;
}

/* Logo / Site adi */
.site-title {
  font-size: var(--text-xl);
  font-weight: 700;
  color: var(--color-primary);
}

/* ===== NAV LISTESI ===== */
nav ul {
  display: flex;
  flex-wrap: wrap;
  list-style: none;
  gap: var(--space-sm);
  padding: 0;
  margin: 0;
}

nav a {
  display: inline-block;
  padding: var(--space-xs) var(--space-md);
  color: var(--color-text);
  text-decoration: none;
  border-radius: var(--radius-md);
  transition: background var(--transition-fast),
              color var(--transition-fast);
}

nav a:hover,
nav a:focus {
  background-color: var(--color-primary);
  color: var(--color-bg);
}

/* ===== MOBiLDE DiKEY YIGIN ===== */
@media (max-width: 639px) {
  header {
    flex-direction: column;
    gap: var(--space-sm);
  }

  nav ul {
    flex-direction: column;
    align-items: center;
    width: 100%;
  }

  nav a {
```

```
    display: block;
    text-align: center;
    width: 100%;
  }
}
```

Uygulama Adımı

Uygulama-5: Toolbar (Beceri Etiketleri)
"Hakkımda" bölümünde kullandığın teknolojileri etiket (tag/badge) şeklinde göster — Flexbox ile sarmalanan bir toolbar:

```
<ul class="skill-tags" role="list"
    aria-label="Beceri etiketleri">
  <li>HTML5</li>
  <li>CSS3</li>
  <li>JavaScript</li>
  <li>React</li>
  <li>TypeScript</li>
  <li>Git</li>
</ul>
```

```
.skill-tags {
  display: flex;
  flex-wrap: wrap;
  gap: var(--space-sm);
  list-style: none;
  padding: 0;
}

.skill-tags li {
  background-color: var(--color-primary);
  color: var(--color-bg);
  padding: var(--space-xs) var(--space-md);
  border-radius: var(--radius-full);
  font-size: var(--text-sm);
}
```

`flex-wrap: wrap` sayesinde etiketler satıra sığmadığında otomatik olarak alt satıra kayar — ekran boyutuna göre düzen kendini ayarlar.

Sırada Ne Var?

Flexbox ile navigasyonu ve toolbar'ı responsive yaptık. Şimdi projelerin kartlarını **Grid** ile ızgara düzenine yerleştireceğiz.

9 Uygulama: Grid ile Kart Düzeni

Portföy sayfandaki "Projelerim" bölümünde proje kartları şu anda dikey olarak alt alta sıralanıyor. Grid ile bu kartları responsive bir ızgara düzenine geçireceğiz.

Uygulama Adımı

Uygulama-6: Responsive Proje Kartları
HTML yapısı (LAB-2'den):

```html
<section id="projeler">
  <h2>Projelerim</h2>
  <div class="project-grid">
    <article class="project-card">
      <img src="proje1.jpg"
           alt="E-Ticaret sitesi anasayfa ekran goruntusu" />
      <h3>E-Ticaret Sitesi</h3>
      <p>React ve Node.js ile gelistirilmis
         tam kapsamli bir e-ticaret uygulamasi.</p>
      <ul class="skill-tags">
        <li>React</li>
        <li>Node.js</li>
        <li>MongoDB</li>
      </ul>
    </article>

    <article class="project-card">
      <img src="proje2.jpg"
           alt="Blog uygulamasi yazi listesi gorunumu" />
      <h3>Blog Uygulamasi</h3>
      <p>Kisisel blog platformu.
         Markdown destekli yazi editoru.</p>
      <ul class="skill-tags">
        <li>TypeScript</li>
        <li>Next.js</li>
      </ul>
    </article>

    <article class="project-card">
      <img src="proje3.jpg"
           alt="Hava durumu uygulamasi arayuzu" />
      <h3>Hava Durumu</h3>
      <p>OpenWeather API ile anlik
         hava durumu bilgisi.</p>
      <ul class="skill-tags">
        <li>JavaScript</li>
        <li>API</li>
      </ul>
    </article>
  </div>
</section>
```

CSS — Grid ile responsive kart düzeni:

```
/* ===== PROJE GRID ===== */
.project-grid {
  display: grid;
  /* Sihirli satir: otomatik responsive grid! */
  grid-template-columns:
    repeat(auto-fit, minmax(280px, 1fr));
  gap: var(--space-xl);
}

/* ===== PROJE KARTI ===== */
.project-card {
  display: flex;
  flex-direction: column;
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  overflow: hidden;
  transition: transform var(--transition-normal),
              box-shadow var(--transition-normal);
}

.project-card:hover {
  transform: translateY(-4px);
  box-shadow: var(--shadow-lg);
}

/* Kart icindeki gorsel */
.project-card img {
  width: 100%;
  height: 200px;
  object-fit: cover;
}

/* Kart icerigi */
.project-card h3,
.project-card p,
.project-card .skill-tags {
  padding: 0 var(--space-lg);
}

.project-card h3 {
  margin-top: var(--space-md);
  font-size: var(--text-lg);
}

.project-card p {
  color: var(--color-muted);
  flex-grow: 1; /* icerik farkli uzunlukta olsa
                   bile kartlarin alt kenari hizali */
```

```
}

.project-card .skill-tags {
  padding-bottom: var(--space-lg);
  margin-top: auto;
}
```

Derinlemesine Anlama

`repeat(auto-fit, minmax(280px, 1fr))` Nasıl Çalışır?
Bu tek satır, media query yazmadan responsive grid sağlar:

1. Tarayıcı, kapsayıcı genişliğine bakar.
2. Her sütun en az 280px olmalı — kaç sütun sığar hesaplar.
3. Kalan boş alanı `1fr` ile sütunlara eşit dağıtır.
4. Ekran daraldığında sütun sayısı 3 → 2 → 1 şeklinde otomatik azalır.

`auto-fit` vs `auto-fill`:

- `auto-fit`: Boş sütun varsa daraltır, mevcut elemanlar genişler.
- `auto-fill`: Boş sütunları korur, elemanlar genişlemez.

Çoğu durumda `auto-fit` istediğin davranışı verir.

Dikkat – Sık Yapılan Hatalar

- Kart içindeki görseller `object-fit: cover` olmadan orantısız şekilde uzar veya sıkışır. Görsel boyutu tutarlı olsun diye her zaman `object-fit` belirle.
- Grid ile kartları eşit yükseklikte yapmak istiyorsan, kart içinde `display: flex; flex-direction: column` ve açıklama paragrafına `flex-grow: 1` ver — kartlar dikey olarak eşitlenir.
- `minmax()` içindeki minimum değeri çok büyük tutma. 400px minimum, mobilde tek sütun bile sığmaz.

Sırada Ne Var?

Grid ile kart düzenini kurduk. Şimdi tüm parçaları birleştirip portföy sayfasını **3 breakpoint'e göre tam responsive** hale getireceğiz.

10 Uygulama: Portföy Sayfasını 3 Breakpoint'e Göre Responsive Yap

Bu bölümde LAB-2'de oluşturduğun portföy sayfasını mobilden masaüstüne kadar 3 farklı düzende çalışır hale getireceksin. Bu, tüm LAB-3 bilgilerinin birleştiği ana uygulama adımıdır.

Uygulama Adımı

Uygulama-7: Responsive Portföy — Tam CSS
Aşağıdaki CSS'i mobil-first mantığıyla uygula:

```
/* ========================================
   MOBIL VARSAYILAN (0 - 639px)
   ======================================== */

/* Genel sifirlama */
*, *::before, *::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  font-family: var(--font-sans);
  font-size: var(--text-base);
  line-height: 1.6;
  color: var(--color-text);
  background-color: var(--color-bg);
}

img {
  max-width: 100%;
  height: auto;
  display: block;
}

/* Section'lar arasi bosluk */
section {
  padding: var(--space-xl) var(--space-md);
}

/* Section basliklari */
section h2 {
  font-size: var(--text-2xl);
  color: var(--color-primary);
  margin-bottom: var(--space-lg);
  text-align: center;
}

/* Hakkimda: tek sutun, ortalanmis */
.about-content {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: var(--space-lg);
  text-align: center;
}
```

```
.about-content figure {
  max-width: 200px;
}

.about-content img {
  border-radius: var(--radius-full);
  aspect-ratio: 1;
  object-fit: cover;
}

/* Iletisim formu */
.form-group {
  margin-bottom: var(--space-md);
}

label {
  display: block;
  margin-bottom: var(--space-xs);
  font-weight: 600;
}

input, textarea, select {
  width: 100%;
  padding: var(--space-sm) var(--space-md);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-md);
  font-family: inherit;
  font-size: var(--text-base);
  transition: border-color var(--transition-fast);
}

input:focus, textarea:focus, select:focus {
  border-color: var(--color-primary);
  outline: 2px solid var(--color-secondary);
  outline-offset: 2px;
}

button[type="submit"] {
  width: 100%;
  padding: var(--space-sm) var(--space-lg);
  background: var(--color-primary);
  color: var(--color-bg);
  border: none;
  border-radius: var(--radius-md);
  font-size: var(--text-base);
  cursor: pointer;
  transition: background var(--transition-fast);
}
```

```
button[type="submit"]:hover,
button[type="submit"]:focus {
  background: var(--color-secondary);
}

/* Footer */
footer {
  text-align: center;
  padding: var(--space-xl) var(--space-md);
  background: var(--color-surface);
  border-top: 1px solid var(--color-border);
  color: var(--color-muted);
  font-size: var(--text-sm);
}

/* ========================================
   TABLET (640px ve uzeri)
   ======================================== */
@media (min-width: 640px) {
  section {
    padding: var(--space-2xl) var(--space-xl);
  }

  /* Hakkimda: yatay duzen */
  .about-content {
    flex-direction: row;
    text-align: left;
    align-items: flex-start;
  }

  .about-content figure {
    flex-shrink: 0;
  }

  /* Form butonu tam genislik olmaktan cikar */
  button[type="submit"] {
    width: auto;
  }
}

/* ========================================
   MASAUSTU (1024px ve uzeri)
   ======================================== */
@media (min-width: 1024px) {
  /* Icerik genisligi sinirla */
  main {
    max-width: 1200px;
    margin: 0 auto;
  }

```

```
  section {
    padding: var(--space-3xl) var(--space-xl);
  }

  /* Proje kartlari en az 3 sutun */
  .project-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Uygulama Adımı

Uygulama-8: Ekran Görüntüleri Al

1. **Chrome DevTools** (F12) → üst menüde "Toggle device toolbar" simgesi (veya `Ctrl+Shift+M`).
2. **Mobil** (375px genişlik) görünümünde tam sayfa ekran görüntüsü al:
   - DevTools açıkken `Ctrl+Shift+P` (komut paleti) → "Capture full size screenshot" yaz → Enter.
   - Dosya adı: `screenshot-mobile.png`
3. **Tablet** (768px genişlik) görünümünde aynı işlemi tekrarla: `screenshot-tablet.png`
4. **Masaüstü** (1280px genişlik) görünümünde: `screenshot-desktop.png`
5. Her üç dosyayı proje klasöründe `screenshots/` dizinine kaydet.

Dikkat – Sık Yapılan Hatalar

- DevTools'un responsive modunda `width` alanına istediğin piksel değerini yazabilirsin. Hazır cihaz listesinden seçme zorunluluğun yok.
- Ekran görüntülerinde sayfanın **tamamı** görünmelidir (scroll dahil). "Capture full size screenshot" bunu yapar.
- CSS'inde `px` yerine `rem` ve `var(--space-*)` kullandığından emin ol. Piksel kullanan layout responsive davranmaz.

Sırada Ne Var?

Portföy sayfası artık 3 breakpoint'te responsive çalışıyor ve ekran görüntülerini aldık. Şimdi "CSS Kararları" notunu yazıp, Git iş akışı ile teslim edeceğiz.

11 CSS Kararları Notu

Teslimde ekran görüntülerinin yanı sıra kısa bir **"CSS Kararları" notu** da isteniyor. Bu not, tasarım sürecindeki bilinçli tercihlerini belgelemendir.

Uygulama Adımı

Uygulama-9: CSS Kararları Notu Yaz

Proje klasöründe `CSS-KARARLARI.md` dosyası oluştur (veya README'ye ekle). Aşağıdaki başlıklarda kısa açıklamalar yaz:

```
# CSS Kararlari

## 1. Breakpoint Secimi
- Neden 640px ve 1024px sectim?
- Icerigim bu noktalarda nasil degisiyor?

## 2. Layout Tercihleri
- Header icin neden Flexbox sectim?
- Proje kartlari icin neden Grid sectim?
- auto-fit mi auto-fill mi kullandim, neden?

## 3. Design Tokens
- Hangi renk paletini sectim ve neden?
- Spacing skalasini nasil belirledim?
- Fluid typography icin clamp degerlerini
  nasil ayarladim?

## 4. Responsive Stratejiler
- Mobile-first yaklasimini nasil uyguladim?
- Hangi elemanlar breakpoint'lerde degisiyor?
- Gorsel boyutlari nasil yonettim?
```

Her madde 2–3 cümle yeterlidir. Amaç, kararların **rastgele değil bilinçli** olduğunu göstermektir.

Sırada Ne Var?

CSS kararları notunu da yazdık. Şimdi tüm çalışmayı Git ile kayıt altına alıp GitHub'a yükleyelim.

12 Git İş Akışı ve Teslim

LAB-1 ve LAB-2'de öğrendiğin Git iş akışını bu LAB'da da uygulayacaksın.

Uygulama Adımı

Uygulama-10: Git ile Kaydet ve GitHub'a Yükle

1. Yeni branch aç:

```
git checkout -b feature/responsive-layout
```

2. Adım adım commit at (her mantıksal değişiklik bir commit):

```
git add src/styles/tokens.css
git commit -m "feat: add CSS design tokens"

git add src/styles/
git commit -m "feat: add responsive navigation with Flexbox"
```

```

git add src/styles/
git commit -m "feat: add project card grid layout"

git add src/styles/
git commit -m "feat: implement 3-breakpoint responsive design
    "

git add screenshots/
git commit -m "docs: add mobile/tablet/desktop screenshots"

git add CSS-KARARLARI.md
git commit -m "docs: add CSS decisions documentation"
```

3. GitHub'a push et:

```
git push -u origin feature/responsive-layout
```

13 Teslim Kontrol Listesi ve Değerlendirme Rubriği

Kontrol Listesi

Teslim öncesi kontrol listesi:

- ☐ CSS değişkenleri (`:root` içinde design tokens) tanımlanmış mı?
- ☐ Fluid typography (`clamp()`) uygulanmış mı?
- ☐ Navigasyon çubuğu Flexbox ile responsive yapılmış mı?
- ☐ Proje kartları Grid ile ızgara düzeninde mi?
- ☐ Mobile-first yaklaşım uygulanmış mı? (`min-width` media query'ler)
- ☐ En az 3 breakpoint tanımlanmış ve çalışıyor mu? (mobil/tablet/masaüstü)
- ☐ Görseller `max-width: 100%` ile responsive mi?
- ☐ `<meta name="viewport">` etiketi mevcut mu?
- ☐ Tab ile gezinme ve focus göstergesi hâlâ çalışıyor mu? (LAB-2 kazanımları bozulmamış)
- ☐ 3 ekran görüntüsü (mobil/tablet/masaüstü) alınmış mı?
- ☐ CSS Kararları notu yazılmış mı?
- ☐ En az 3 anlamlı commit atılmış ve branch kullanılmış mı?
- ☐ GitHub'a push edilmiş mi?

13.1 Değerlendirme Rubriği

Kriter	Puan	Açıklama
CSS değişkenleri (design tokens) tanımlı	10	:root içinde renk, spacing, radius
Fluid typography uygulanmış	10	clamp() ile akıcı font boyutları
Flexbox ile nav/toolbar	15	Responsive navigasyon çubuğu

Kriter	Puan	Açıklama
Grid ile kart düzeni	15	Responsive proje kartları ızgarası
Mobile-first 3 breakpoint	15	Mobil/tablet/masaüstü düzenleri çalışıyor
Görseller responsive	5	max-width: 100%, object-fit
Ekran görüntüleri (3 adet)	10	Mobil, tablet, masaüstü tam sayfa
CSS Kararları notu	10	Breakpoint, layout, token tercihleri açıklanmış
Git iş akışı (branch + commit)	10	En az 3 anlamlı commit
Toplam	**100**	

14 Kendini Test Et

Kendini Test Et

Aşağıdaki sorulara kendi cümlelerinle 2–3 satır cevap yazabiliyor musun? Yazamıyorsan ilgili bölümü tekrar oku.

1. **Mobile-first yaklaşım nedir? `min-width` ve `max-width` farkı nedir?**
2. **Breakpoint nedir ve nasıl belirlenir? İçeriğe göre mi, cihaz modeline göre mi?**
3. **CSS custom property nasıl tanımlanır ve nasıl kullanılır? `var()` fonksiyonunun fallback değeri ne işe yarar?**
4. **Design token nedir? Neden renk ve spacing değerlerini tek bir yerde toplarız?**
5. **`clamp()` fonksiyonunun 3 parametresi nedir? Neden sadece `vw` kullanmak erişilebilirlik sorununa yol açar?**
6. **Flexbox ile Grid arasındaki temel fark nedir? Hangisini ne zaman kullanırsın?**
7. **`justify-content` ve `align-items` arasındaki fark nedir?**
8. **`repeat(auto-fit, minmax(280px, 1fr))` ne yapar? Neden media query'ye gerek kalmaz?**
9. **`fr` birimi nedir? `1fr 2fr 1fr` ne anlama gelir?**
10. **`gap` özelliği neden `margin` kullanmaktan daha pratiktir?**

15 Ek Kaynaklar ve İleri Okuma

- **Flexbox Rehberi:** https://css-tricks.com/snippets/css/a-guide-to-flexbox/ — CSS-Tricks'in kapsamlı görsel Flexbox rehberi.
- **Grid Rehberi:** https://css-tricks.com/snippets/css/complete-guide-grid/ — CSS-Tricks'in kapsamlı görsel Grid rehberi.

- **Flexbox Froggy:** https://flexboxfroggy.com — Flexbox öğrenmek için interaktif oyun.
- **Grid Garden:** https://cssgridgarden.com — CSS Grid öğrenmek için interaktif oyun.
- **Responsive Design (MDN):** https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Responsive_Design
  MDN responsive tasarım rehberi.
- **Modern CSS Solutions:** https://moderncss.dev — Stephanie Eckles'ın modern CSS teknikleri koleksiyonu.
- **Every Layout:** https://every-layout.dev — Reusable CSS layout kalıpları.
- **Utopia (Fluid Type):** https://utopia.fyi — Fluid typography ve spacing hesaplayıcı.
- **Design Tokens (W3C):** https://www.w3.org/community/design-tokens/ — Design token'lar W3C topluluk grubu.